

GitHub Copilot CLI Feature Research and Training Playbook

Executive summary

GitHub Copilot CLI reached **general availability on February 25, 2026**, and (as of late February 2026) it has evolved from a “terminal chat helper” into a **terminal-native agentic environment** that can **plan, edit files, run commands/tests, review diffs, parallelize work with subagents, and retain session/repository memory across sessions**—all while keeping humans in control through **trust prompts, tool/path/URL permissions**, and enterprise **policy + audit logging**. ¹

For an internal training program (live demos + 45–90s shorts + 3–6 minute tutorials + labs), the best payoff comes from teaching a **repeatable workflow** rather than a list of commands:

- 1) **Scope** (trusted folder + limited allowed tools)
- 2) **Plan** (plan mode + explicit acceptance criteria)
- 3) **Execute safely** (interactive approvals or autopilot with guardrails)
- 4) **Verify** (tests + `/diff` + `/review`)
- 5) **Share/continue** (sessions, Markdown/gist sharing, delegation, CI automation)

This workflow maps directly to the platform’s newest high-value capabilities: **plan mode, autopilot mode, /fleet parallelism, /diff + Esc–Esc rewind, /review, model selection + cost visibility**, plus enterprise-grade customization via **custom agents, skills, hooks, plugins, MCP servers**, and advanced integration via **ACP** and the **Copilot SDK**. ²

Audience mix is **unspecified**, so the recommendations below prioritize: (a) universal terminal workflows (Git, tests, refactor, review), (b) governance patterns that work for mixed dev/QA/DevOps groups, and (c) demo designs that are stable across stacks. ³

Timeline and recent releases

Copilot CLI’s late-2025 → early-2026 release train created several “must-demo” inflection points: deprecations/migration, model selection, parallelism, policy hooks, and GA-grade durability. ⁴

```
timeline
  title Copilot CLI milestones (public preview → GA)
  2025-09-25 : Deprecation announced for gh-copilot extension (stop 2025-10-25)
  2025-10-03 : /model + image recognition + "!" direct shell execution
  2025-10-28 : Custom agents + /delegate + streaming + parallel tool calling
  2025-11-18 : Bundled ripgrep + grep/glob tools + /share + headless scripting
```

improvements

2026-01-14 : Built-in specialized agents + stronger context management + new install paths

2026-01-21 : Plan mode improvements + tighter GitHub CLI integration (gh copilot)

2026-01-28 : ACP support (copilot --acp) for third-party integrations/automation

2026-02-25 : Copilot CLI generally available + /diff + /review + Esc-Esc rewind + memory + enterprise readiness

2026-02-27 : “From idea to PR” training-focused workflow guide published

Key migration context: the **legacy** `gh-copilot` **GitHub CLI extension stopped functioning October 25, 2025**, and GitHub positioned the new Copilot CLI as its replacement. ⁵

Key “fresh” sources for your training hub (late Feb 2026): GA announcement and the new long-form workflow blog published **February 27, 2026**. ⁶

High-value features to teach and demo

Feature priority matrix for live demos

The table below optimizes for: (1) wow factor, (2) repeatability for 45–90s shorts, (3) teachability with guardrails, and (4) low risk of org-specific setup surprises. The commands/shortcuts are taken from the official command reference and recent GitHub changelog posts. ⁷

Feature	What it does	Best demo entry point	Prep required	Est. demo time	Priority
Plan mode	Creates a structured plan before edits/commands	<code>Shift+Tab</code> → plan mode, or <code>/</code> plan	Small repo with a clear task	4–6 min	High
Autopilot mode	Continues multi-step work without back-and-forth prompts	<code>Shift+Tab</code> → autopilot mode; or <code>copilot --autopilot</code> in prompt mode	Decide permission posture (<code>--allow-all</code> vs limited)	4–7 min	High
<code>/diff</code> + Undo/rewind	Review session diffs; rewind snapshots with <code>Esc-Esc</code>	<code>/diff</code> , then <code>Esc</code> twice to rewind	Repo with a small change	3–6 min	High

Feature	What it does	Best demo entry point	Prep required	Est. demo time	Priority
<code>/review</code>	Runs code review agent on staged/unstaged changes	<code>/review</code>	A deliberate bug/smell in code	3–5 min	High
Permissions (tools/paths/URLs)	Fine-grained allow/deny patterns for safety	<code>--allow-tool</code> , <code>--deny-tool</code> , <code>/add-dir</code> , <code>/list-dirs</code>	Decide a “safe tool allowlist”	5–8 min	High
Programmatic/headless (<code>-p</code> , <code>-s</code>)	Run Copilot CLI in scripts; return just output	<code>copilot -p "..." -s</code>	Terminal environment + auth token for automation	2–4 min	High
<code>/model</code> + <code>/usage</code> + <code>/context</code>	Pick models; show cost and token usage	<code>/model</code> , <code>/usage</code> , <code>/context</code>	Ensure enterprise model policy allows options	3–6 min	High
<code>/fleet</code> parallel subagents	Breaks a plan into parallelizable subtasks	<code>/fleet <prompt></code>	Task that splits naturally (tests + docs + refactor)	5–9 min	Medium
Sessions (<code>--continue</code> , <code>/resume</code>)	Resume long sessions; keep context	<code>copilot --continue</code> / <code>/resume</code>	Run at least one prior session	2–4 min	Medium
Custom agents	Specialized agent personas (tools/models)	<code>/agent</code> or <code>--agent</code>	One <code>.agent.md</code> in repo	6–10 min	Medium
Skills	Load specialized workflows from <code>SKILL.md</code>	<code>/skills</code> management	One simple skill (“run tests + summarize”)	6–10 min	Medium
Hooks	Policy enforcement + auditing at lifecycle points	<code>.github/hooks/*.json</code> + <code>preToolUse</code>	Prebuilt <code>hooks</code> example scripts	8–12 min	Medium
Plugins	Install reusable bundles (skills/agents/hooks/MCP)	<code>copilot plugin install ...</code>	A known-good plugin repo	5–10 min	Low–Medium

Feature	What it does	Best demo entry point	Prep required	Est. demo time	Priority
MCP servers	Extend tool access (GitHub built-in; add more)	<code>/mcp add</code>	MCP endpoint + policy decision	6–12 min	Low–Medium
<code>/share</code> + gist sharing	Export sessions to Markdown or gist	<code>/share file</code> or <code>/share gist</code>	Auth + desire to share logs	1–3 min	Low
ACP mode (<code>--acp</code>)	Run Copilot as an agent server for other clients	<code>copilot --acp</code>	Only if you have an ACP client or demo script	6–12 min	Low
Copilot SDK	Programmatic multi-turn + tools from code	SDK in Node/Python/Go/.NET	Dev environment + sample code	8–15 min	Low

Demo scripts for the top features

The scripts below are designed as **copy/paste runbooks** for live training. “Expected output” describes **stable UI behaviors** (what you can reliably point to), not exact text. Exact content will vary by repo/task/model. ⁸

Demo module: Plan → build loop that teaches safe agentic work

Why it’s worth teaching: Plan mode reduces “AI surprise” by forcing alignment before edits. This is highlighted as a core GA workflow and is reinforced in official docs. ⁹

Setup (1–2 min): - Use a small repo with one failing test or a tiny feature request. - Start in the repo root (not in `$HOME`). GitHub docs explicitly warn against launching from home directories. ¹⁰

Commands / keystrokes:

```
copilot
# Shift+Tab -> plan mode
/plan Add a new CLI flag --json that prints output as JSON, and add tests.
```

Expected behaviors to narrate: - Copilot asks clarifying questions, then presents a **structured plan** before code changes. ¹¹ - You can **accept/iterate** on the plan; acceptance becomes a “contract” for execution. ¹²

Teaching points: - Add explicit acceptance criteria to the plan (“tests green”, “docs updated”, “no breaking changes”). This aligns strongly with official best-practice framing around planning and delegation. ¹³

Demo module: Autopilot mode with guardrails

Autopilot mode lets Copilot continue without waiting for user prompts, but it's paired with explicit warnings: permission posture + premium request usage can "advance" without the user typing each step. ¹⁴

Safe baseline approach for training: demonstrate autopilot **with a constrained permission set** first, then show the difference when enabling all permissions.

Commands / keystrokes:

```
copilot --allow-all --max-autopilot-continues 10
# Shift+Tab -> plan mode, create plan, then choose "Accept plan and build on
autopilot" when offered
```

Or programmatic autopilot in prompt mode:

```
copilot --autopilot --allow-all -p "Fix the failing unit tests and run them."
```

Expected behaviors: - Copilot shows it's continuing autonomously and indicates premium request usage per continuation step. ¹⁵ - If you didn't grant all permissions, autopilot prompts you to either enable permissions, continue limited, or cancel. ¹⁶

Key caution to teach explicitly: - In autopilot, granting full permissions effectively gives Copilot the ability to do what you can do (files/commands/URLs), so the safe posture is: start minimal; expand with intent; keep `--max-autopilot-continues` bounded. ¹⁷

Demo module: `/diff` + `/review` + rewind snapshots

This bundle is the highest leverage "trust builder" for teams: it demonstrates that even when Copilot edits multiple files, review/undo is first-class. ¹⁸

Commands / keystrokes:

```
/diff
/review
# Undo/rewind:
Esc Esc
```

Expected behaviors: - `/diff` shows **syntax-highlighted inline diffs** and can toggle between session changes and branch diffs (per GA notes). ¹⁸ - `/review` analyzes staged/unstaged changes (described both in Jan 2026 improvements and GA highlights). ¹⁹ - `Esc-Esc` provides "undo/rewind" to a previous snapshot within the session. ²⁰

Critical gotcha to teach (with a mitigation): - A recent open bug report (Feb 25, 2026) alleges that restoring to checkpoint can run `git clean -fd`, deleting **untracked files**. For training, mitigate by using a clean repo, ensuring generated artifacts are `.gitignore`'d, or committing/branching before risky agent runs. ²¹

Demo module: Permissions that scale to enterprise training

This is where most org rollouts either succeed safely or fail noisily. The CLI supports: - tool allow/deny patterns

- URL allow/deny
- path scoping (`/add-dir` , `--add-dir`)
- “allow all” shortcuts (`--allow-all` , `--yolo` , `/yolo`)
- deny precedence, even under allow-all (important!) ²²

Suggested “training” policy posture: - Allow safe read/search + test commands; deny destructive commands; deny push by default; require explicit human action for releases.

Concrete command examples:

```
# Allow git commands but deny pushes
copilot --allow-tool 'shell(git:*)' --deny-tool 'shell(git push)'

# Restrict tool surface drastically (teaching moment: "less is more")
copilot --available-tools 'read' 'grep' 'glob' 'shell(git status)' 'shell(npm
test)'
```

```
# URL governance
copilot --allow-url 'github.com' --deny-url 'https://*.example.com'
```

Expected behaviors: - Copilot will not prompt for allowed patterns; will block/ask for denied ones; deny rules override allow rules. ²³

Demo module: Programmatic mode for scripts and “AI utilities”

Official docs present `-p/--prompt` as the core way to use Copilot CLI non-interactively, and `-s/--silent` for script-friendly output. ²⁴

Commands:

```
# One-off scripted ask
copilot -p "Summarize the last 10 commits into a release note draft."
```

```
# Script-friendly: only the response
```

```
copilot -sp "Generate a bash one-liner to list the 10 largest files in this repo."
```

Expected behaviors: - Prints response and exits; `-s` suppresses extra UI/usage details (supports piping).

²⁴ - In automation scenarios, Copilot CLI has specific auth env vars (e.g., `COPILOT_GITHUB_TOKEN`). ²⁵

Security, governance, and safe enablement

Trust boundaries: folders, tools, paths, URLs, and memory

Copilot CLI has two separate but complementary “trust” concepts:

Directory trust (file scope): On session start, users must confirm they trust the directory contents; persistent trust can be recorded in config. ²⁶

Tool approvals (execution scope): When Copilot needs to execute a tool (like shell commands, edits), it requests approval with options to allow once, allow for the session, or deny and steer. ²⁷

Training guidance that aligns with GitHub docs: - Do not start sessions from directories that may contain untrusted executables or sensitive/confidential data—GitHub explicitly cautions against launching from the home directory. ¹⁰ - Teach a strict progression: **read-only exploration** → **plan** → **limited execution** → **broader autonomy**, only when the folder and task are suitable. ²⁸

Enterprise policy and audit logging

Enterprise owners can control Copilot CLI usage via **AI controls** and client policies; model availability in CLI reflects enterprise model enablement. ²⁹

There are also important “policy edges” to teach:

- `/delegate` requires **both Copilot CLI and Copilot coding agent policies enabled**. ³⁰
- Copilot CLI usage appears in **enterprise audit logs**. ²⁹
- Some enterprise AI controls do **not** apply to Copilot CLI (notably, the enterprise doc calls out MCP server policies and content exclusions as not affecting CLI). This is a governance discussion you may need to escalate with security/IT, because it affects how teams assume controls work. ²⁹

Hooks as “policy-as-code” for Copilot CLI

Hooks are the official mechanism to enforce process at lifecycle points (session start/end, prompt submission, before/after tool execution). Hook files load from `.github/hooks/*.json`. ³¹

The key governance lever is `preToolUse`, which can: - allow, deny, or ask

- block dangerous patterns (e.g., `rm -rf`, `git push`)
- modify tool arguments (e.g., force `--dry-run`) ³²

This is ideal for enterprise training because you can ship one repo that demonstrably enforces your policies and produces local audit logs (as shown in the official hooks tutorial). ³³

Recommended “minimum viable guardrails” for a mixed audience training

A pragmatic baseline that maps to the CLI’s permission primitives:

- **Allowed tools:** `read`, `grep`, `glob`, safe `shell(git status)`, safe build/test commands. ²³
- **Denied tools:** destructive shells (`rm`, `chmod`, `chown`), `git push` unless explicitly allowed. ²³
- **Path scoping:** use `/add-dir` for a sandbox directory, and teach `/list-dirs`. ³⁴
- **URL scoping:** use `--allow-url` only for known domains; deny unknown/unsafe by default. ²³
- **Snapshot safety:** warn users that session rewind (Esc–Esc) is powerful but may have side effects on untracked files in some versions; teach `.gitignore` hygiene + “commit early” patterns. ³⁵

Models, configuration, and cost controls

Model selection and premium requests

Copilot CLI supports switching models mid-session via `/model` (and `--model / COPILOT_MODEL`), and GitHub explicitly positions “models, your choice” as a headline capability. ³⁶

Premium request accounting is essential to teach because it affects budget and user behavior:

- GitHub’s billing docs state that **each Copilot CLI prompt uses premium requests**, and model selection can apply multipliers. ³⁷
- Autopilot mode can consume additional premium requests during continuations without the user typing each step; GitHub’s autopilot documentation calls this out explicitly. ¹⁶
- Copilot CLI provides `/usage` and `/context` to expose session usage, token breakdown, and premium request consumption (useful for teaching “cost awareness”). ³⁸

Configuration file locations and precedence

This is one of the most practical pieces for internal trainers because it makes setups reproducible:

- User config defaults to `~/.copilot/config.json` (Windows: `$HOME\.copilot\config.json`). ³⁹
- The config directory can be overridden with `XDG_CONFIG_HOME` (also affects MCP config storage). ⁴⁰
- Settings cascade across scopes: user `~/.copilot/config.json`, project `.copilot/settings.json`, local `.copilot/settings.local.json` (recommended to gitignore). Flags/env override everything. ²³

Trainer-friendly options worth highlighting: - `streamer_mode`: hides preview model names and quota details—useful when recording short videos. ⁴¹

- `reasoning_effort`: dial “thinking intensity” (and potentially productivity/cost). ⁴²
- `trusted_folders`: pre-authorize safe workspaces for training labs. ³⁹

Environment variables that matter in practice

The command reference provides a definitive list; the most training-relevant ones are:

- `COPILOT_GITHUB_TOKEN`, `GH_TOKEN`, `GITHUB_TOKEN` for auth precedence (avoid confusion in CI). ²⁵
- `COPILOT_EDITOR` (checked after `$VISUAL` and `$EDITOR`) to control editor invocation for longer prompt editing; this resolves a common “why did it open vi?” complaint. ⁴¹
- `COPILOT_ALLOW_ALL` to allow all permissions automatically (equivalent to `--allow-all`). ⁴³
- `USE_BUILTIN_RIPGREP` to control bundled vs system `ripgrep` (useful in locked-down environments). ⁴⁴

Advanced workflows for power users

Automation patterns that are “officially supported”

Headless scripting: GitHub’s docs and blog both push `copilot -p` for automation and scripting, with `-s` for clean output. ⁴⁵

GitHub Actions integration: There is now an official guide for running Copilot CLI in Actions, including: - install (e.g., npm)

- authenticate using a fine-grained PAT with the **Copilot Requests** permission

- run `copilot -p` in a job step ⁴⁶

A stable “teachable” use case is a scheduled “daily summary” job (official example), because it is read-only and easy to verify in logs. ⁴⁶

```
flowchart TD
    A[Trigger: schedule or workflow_dispatch] --> B[Checkout repo]
    B --> C[Install Copilot CLI]
    C --> D[Auth via COPILOT_GITHUB_TOKEN secret]
    D --> E[Run copilot -p in programmatic mode]
    E --> F[Use output: logs, PR comment, artifact, report]
```

Scaling customization: instructions vs skills vs agents vs plugins

The “Comparing CLI features” doc is the most useful official framework for teaching customization without confusion: it distinguishes **custom instructions** (general behavior), **skills** (task-specific enhancements), **tools/MCP servers** (capabilities), **hooks** (lifecycle enforcement), **subagents** (isolated contexts), **custom agents** (specialized personas), and **plugins** (distribution mechanism). ⁴⁷

A trainer-friendly “layering strategy”: - Begin with `/init` or `copilot init` to seed baseline instructions. ⁴²

- Add 1–2 org-wide skills (test runner, PR prep) in `.github/skills/...` to avoid repeating prompts. ⁴⁸

- Add hooks for “non-negotiables” (deny `git push`, block secrets, require tests). ⁴⁹
- Package it as a plugin for reuse if you need one installable artifact. ⁵⁰

Parallelism and subagent performance with `/fleet`

`/fleet` is positioned as the official parallelization feature: the main agent decomposes and orchestrates subtasks executed by subagents with separate context windows. ⁵¹

- What to teach: - Use `/fleet` only when work truly decomposes (tests/docs/refactor across files). ⁵²
- Set expectations: `/fleet` can increase premium request usage because multiple subagents can interact with models. ⁵³
 - Teach “plan first”, “fleet second”: the docs describe plan → fleet → autopilot as a typical flow. ⁵⁴

Integration and extensibility: MCP, ACP, and the Copilot SDK

MCP servers: Copilot CLI includes built-in servers and supports persistent MCP config in `~/.copilot/mcp-config.json`, plus session-only MCPs via `--additional-mcp-config`. ⁵⁵

The built-in MCP list includes `github-mcp-server`, `playwright`, `fetch`, and `time` (per command reference). ⁴²

ACP: Copilot CLI implements ACP; you can run `copilot --acp` (stdio) or `copilot --acp --port 8080` (TCP), enabling third-party tools and CI/CD systems to integrate with agentic capabilities. ⁵⁶

Copilot SDK: GitHub Community announced a technical preview Copilot SDK (Node/TS, Python, Go, .NET) that provides consistent APIs for multi-turn conversations, tool execution, and lifecycle control. This is valuable for internal platform teams building approved “Copilot utilities.” ⁵⁷

Migration notes and common “gotchas”

Migration: `gh-copilot` is dead; “`gh copilot`” is new. - The `gh-copilot` extension stopped working **October 25, 2025**. ⁵

- GitHub CLI users can now run `gh copilot` to install/run the new Copilot CLI, forwarding args/flags. ⁵⁸

Gotcha: checkpoint restore and untracked file deletion risk - GA notes highlight “undo/rewind” via `Esc-Esc`. ²⁰

- A recent open issue alleges checkpoint restore can delete untracked files via `git clean -fd`. Teach mitigations: `.gitignore` hygiene, commit/branch early, isolate output directories. ²¹

Gotcha: editor choice - The CLI uses `$VISUAL`, `$EDITOR`, then `COPILLOT_EDITOR`, defaulting to `vi` if none set. Teach `COPILLOT_EDITOR` for predictable team setups and recorded demos. ⁴¹

Gotcha: plan availability vs plan coverage - Plan and autopilot modes are toggled via `Shift+Tab`. ⁴²

- Autopilot differs from `--no-ask-user`; the docs clarify that `--no-ask-user` suppresses clarifying questions but does not replicate full multi-step autonomous continuation. ²⁸

Training asset kit: demos, shorts, blogs, labs, prompt pack

Suggested short-video topics (45–90 seconds)

These are intentionally “single concept” clips; each maps to a supported command/behavior.

- Plan mode in 60 seconds (`Shift+Tab` , `/plan`) 42
- Autopilot guardrails (`--max-autopilot-continues` , permission prompt) 28
- `/diff` + Esc–Esc rewind as a safety net 20
- `/review` before commit: staged vs unstaged 59
- “Scripting Copilot”: `copilot -sp` in a shell pipeline 24
- “Cost visibility”: `/model` + `/usage` in under a minute 60
- “Safer autonomy”: allow git but deny push (`--allow-tool shell(git:*)` + `--deny-tool shell(git push)`) 23
- “Parallelize work”: `/fleet` on a plan (tests + docs + refactor) 52
- “Export your trail”: `/share file` to Markdown 44
- “Enterprise readiness in 30s”: AI controls, model availability, audit logs 61
- “Install path for GH CLI users”: `gh copilot` is the new on-ramp 58
- “Hooks as guardrails”: `preToolUse` denies a risky command 32

Bite-sized blog tutorial outlines (3–6 minutes each)

Each outline below is intentionally small enough to publish as an internal post with 1–2 gifs/screenshots.

- **The “Plan → Execute → Verify” loop in Copilot CLI** (plan mode, autopilot option, `/diff` , `/review`) 6
- **Safe permissions that still feel fast** (allow tool patterns; deny precedence; `/add-dir`) 62
- **Scripting and automation** (programmatic `-p` + `-s` ; exit codes; use cases) 63
- **Copilot CLI in GitHub Actions** (daily summary example; auth best practices) 46
- **Customization decision guide** (instructions vs skills vs hooks vs agents vs plugins) 64
- **Migration: from gh-copilot to Copilot CLI** (what broke; what replaces it; `gh copilot` entry point) 65
- **Parallelizing work responsibly** (`/fleet` + cost implications + task suitability) 53
- **Hooks for compliance** (`preToolUse` gating + logging; ship policy config in repo) 66

Hands-on labs (step-by-step) with learning objectives

These labs are designed to be run in a “training sandbox repo” so the entire class can follow safely and you can reset easily.

Lab	Objective	Time	Artifact
Lab A	Start safely: trust folder + first prompt + stop/cancel	15–20m	Trusted folder set; first session
Lab B	Plan mode: write a plan with acceptance criteria	20–30m	A reviewed plan

Lab	Objective	Time	Artifact
Lab C	Autopilot with limits: implement a small change + tests	25–40m	Code change + tests green
Lab D	Review & rewind: <code>/diff</code> , <code>/review</code> , Esc-Esc snapshot rollback	20–30m	A reviewed diff; rollback familiarity
Lab E	Permissions as code: allow git but deny push; restrict tool surface	20–35m	A repeatable permission config
Lab F	Parallelize: <code>/fleet</code> across tests/docs/refactor	30–45m	Parallel execution experience
Lab G	Hooks: block risky commands via <code>preToolUse</code>	35–55m	Hook config committed in repo
Lab H	CI automation: run Copilot CLI in GitHub Actions to generate summary	35–60m	Workflow file + log output

Lab A: Safe start and trust fundamentals

Learning objectives: understand trust prompt; understand cancel/exit; understand folder scope. ⁶⁷

Steps: 1) Create or clone a small training repo; `cd` into repo root (not `$HOME`). ¹⁰

2) Run:

```
copilot
```

3) When prompted, trust the folder for this session only (recommended for labs). ⁶⁸

4) Ask: “Give me an overview of this project.” (official example style). ⁶⁹

5) Demonstrate exit/cancel: `Esc`, `Ctrl+C`, `/exit`. ⁴¹

Lab B: Plan mode with acceptance criteria

Learning objectives: prompt structuring; acceptance criteria; scope clarity. ⁷⁰

Steps: 1) Start `copilot`

2) `Shift+Tab` to plan mode

3) `/plan` a small change (example: “Add `--json` output mode; add tests; update README.”) ⁴²

4) Refine plan until it includes: test command, files touched, expected behavior.

Lab C: Autopilot with bounded continues

Learning objectives: autonomy mechanics; safety tradeoffs; premium request awareness. ⁷¹

Steps: 1) Run:

```
copilot --allow-all --max-autopilot-continues 6
```

- 2) Plan the task in plan mode; accept plan and build on autopilot. ¹⁴
- 3) Observe the continuation messages and stop with `Ctrl+C` if needed. ²⁸

Lab D: Verify and rewind

Learning objectives: review ergonomics; undo safety; understand untracked file risk.

Steps: 1) After Copilot edits, run:

```
/diff  
/review
```

- 2) Trigger rewind:

```
Esc Esc
```

- 3) Discuss the untracked files risk and mitigation strategies; show `.gitignore` best practices. ³⁵

Lab E: Permission design using patterns

Learning objectives: craft allow/deny rules; understand deny precedence; keep autonomy safe. ²³

Steps: 1) Launch with:

```
copilot --allow-tool 'shell(git:*)' --deny-tool 'shell(git push)'
```

- 2) Ask Copilot to run safe git queries; verify it runs without prompting.
- 3) Ask Copilot to push; verify it is denied.
- 4) Add path scoping with `/add-dir`, list with `/list-dirs`. ³⁴

Lab F: Parallelize with `/fleet`

Learning objectives: learn “parallelizable task” framing; understand cost implications. ⁵³

- Steps: 1) Create a plan that naturally decomposes (tests + docs + refactor).
2) Run:

```
/fleet Implement the plan. Split into parallel tasks: tests, docs, refactor.  
Summarize results and run tests.
```

3) Verify each subagent's output and final merged behavior. 52

Lab G: Hooks for policy enforcement

Learning objectives: lifecycle control; preToolUse gating; auditable behavior. 72

Steps (aligned to official tutorial structure): 1) Create directories:

```
mkdir -p .github/hooks/scripts
mkdir -p .github/hooks/logs
echo ".github/hooks/logs/" >> .gitignore
```

2) Create `.github/hooks/copilot-cli-policy.json` with `sessionStart`, `userPromptSubmitted`, `preToolUse`. 33

3) Implement `pre-tool-policy.sh` to deny `git push` (or deny `rm -rf`).

4) Run `copilot` and demonstrate a blocked tool attempt.

Lab H: Copilot CLI inside GitHub Actions

Learning objectives: safe automation; token handling; stable use cases. 73

Steps: 1) Create a workflow modeled on the official "daily summary" example. 46

2) Use `COPILOT_GITHUB_TOKEN` from a repo secret; ensure the PAT has appropriate Copilot permissions. 73

3) Run on `workflow_dispatch` and inspect logs.

Sample prompts and cheat-sheet (trainer-ready)

One-page cheat-sheet prompts (high reliability)

- "Summarize this repo: architecture, entry points, test command, and how to run locally." 69
- "Create an implementation plan first. Ask clarifying questions. Don't edit files yet." 11
- "Make the smallest diff that fixes the failing tests. Run tests. Show `/diff` and explain changes." 18
- "Run `/review` and only report high-confidence issues." 74
- "Before any destructive command, stop and ask me." (pairs with allow/deny patterns) 23

Ten ready-to-use prompt templates (copy/paste)

1) Plan-first template

"Enter plan mode. Propose a numbered plan with acceptance criteria, test command, and rollback steps. Do not modify files until I approve."

2) Minimal-diff bug fix

"Fix the bug described below with the smallest possible changes. Add or update tests. Run tests. Then show `/diff` and summarize the risk."

3) Explain-before-execute

"Before running any command, explain what you will run and why. Ask for approval if it changes files or could be destructive."

4) Safe refactor

"Refactor `X` for readability without changing behavior. Keep diffs small. Run formatting and tests. Summarize the diff."

5) Upgrade dependency

"Upgrade dependency `X` to the latest allowed version. Identify breaking changes from our code usage (search for imports/usages). Update code and tests. Provide `/diff`."

6) Test generation

"Generate unit tests for `X`. Prioritize boundary conditions and error paths. Run tests and fix failures."

7) Pre-commit review

"Run `/review` on unstaged changes. Only surface high-confidence correctness/security problems and provide concrete fixes."

8) Parallelize with `/fleet`

"Create a plan, then use `/fleet` to parallelize: (a) write tests, (b) update docs, (c) refactor implementation. Merge results and run full test suite."

9) Session cost awareness

"After each major step, run `/usage` and explain what increased token usage or premium requests and how to reduce it (compact context, smaller model, fewer tools)."

10) CI-friendly prompt

"Assume this will run in GitHub Actions. Use programmatic mode. Output plain text suitable for logs. Fail if you can't complete without extra input."

Sources and links

Primary sources were prioritized: GitHub ⁷⁵ Docs, GitHub Blog/Changelog, and official reference pages; supplemented by recent community practices where noted.

Core official documentation and references

```
https://docs.github.com/en/copilot/reference/cli-command-reference
https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-getting-started
https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli
https://docs.github.com/en/copilot/concepts/agents/copilot-cli/about-copilot-cli
https://docs.github.com/en/copilot/concepts/agents/copilot-cli/autopilot
https://docs.github.com/en/copilot/concepts/agents/copilot-cli/fleet
https://docs.github.com/en/copilot/how-tos/copilot-cli/speeding-up-task-
```

completion
<https://docs.github.com/en/copilot/reference/cli-plugin-reference>
<https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/use-hooks>
<https://docs.github.com/en/copilot/tutorials/copilot-cli-hooks>
<https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/configure-copilot-cli>
<https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-with-actions>
<https://docs.github.com/en/copilot/how-tos/copilot-cli/administer-copilot-cli-for-your-enterprise>
<https://docs.github.com/en/copilot/concepts/billing/copilot-requests>
<https://docs.github.com/en/billing/concepts/product-billing/github-copilot-premium-requests>

GitHub changelog and blog posts used

<https://github.blog/changelog/2026-02-25-github-copilot-cli-is-now-generally-available/>
<https://github.blog/ai-and-ml/github-copilot/from-idea-to-pull-request-a-practical-guide-to-building-with-github-copilot-cli/>
<https://github.blog/ai-and-ml/github-copilot/a-cheat-sheet-to-slash-commands-in-github-copilot-cli/>
<https://github.blog/changelog/2026-01-14-github-copilot-cli-enhanced-agents-context-management-and-new-ways-to-install/>
<https://github.blog/changelog/2026-01-21-install-and-use-github-copilot-cli-directly-from-the-github-cli/>
<https://github.blog/changelog/2026-01-28-acp-support-in-copilot-cli-is-now-in-public-preview/>
<https://github.blog/changelog/2025-10-03-github-copilot-cli-enhanced-model-selection-image-support-and-streamlined-ui/>
<https://github.blog/changelog/2025-10-28-github-copilot-cli-use-custom-agents-and-delegate-to-copilot-coding-agent/>
<https://github.blog/changelog/2025-11-18-github-copilot-cli-new-models-enhanced-code-search-and-better-image-support/>
<https://github.blog/changelog/2025-09-25-upcoming-deprecation-of-gh-copilot-cli-extension/>
<https://github.blog/ai-and-ml/github-copilot/power-agentic-workflows-in-your-terminal-with-github-copilot-cli/>

Community discussions and practical guides referenced

<https://github.com/orgs/community/discussions/184872>
<https://github.com/github/copilot-cli/issues/1675>
<https://dev.to/pwd9000/github-copilot-cli-a-devops-engineers-practical-guide-to-ai-powered-terminal-automation-1jh0>

<https://developer.microsoft.com/blog/bringing-work-context-to-your-code-in-github-copilot>

1 2 3 6 8 9 11 12 18 20 35 36 70 <https://github.blog/changelog/2026-02-25-github-copilot-cli-is-now-generally-available/>

<https://github.blog/changelog/2026-02-25-github-copilot-cli-is-now-generally-available/>

4 5 65 [Upcoming deprecation of gh-copilot CLI extension](https://github.blog/changelog/2025-09-25-upcoming-deprecation-of-gh-copilot-cli-extension/?utm_source=chatgpt.com)

https://github.blog/changelog/2025-09-25-upcoming-deprecation-of-gh-copilot-cli-extension/?utm_source=chatgpt.com

7 22 23 25 31 32 34 41 42 43 44 49 55 62 74 <https://docs.github.com/en/copilot/reference/cli-command-reference>

<https://docs.github.com/en/copilot/reference/cli-command-reference>

10 27 [About GitHub Copilot CLI - GitHub Docs](https://docs.github.com/copilot/concepts/agents/about-copilot-cli)

<https://docs.github.com/copilot/concepts/agents/about-copilot-cli>

13 [Best practices for GitHub Copilot CLI](https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-best-practices?utm_source=chatgpt.com)

https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-best-practices?utm_source=chatgpt.com

14 15 16 17 28 71 [Allowing GitHub Copilot CLI to work autonomously - GitHub Docs](https://docs.github.com/en/copilot/concepts/agents/copilot-cli/autopilot)

<https://docs.github.com/en/copilot/concepts/agents/copilot-cli/autopilot>

19 59 [GitHub Copilot CLI: Plan before you build, steer as you go](https://github.blog/changelog/2026-01-21-github-copilot-cli-plan-before-you-build-steer-as-you-go/?utm_source=chatgpt.com)

https://github.blog/changelog/2026-01-21-github-copilot-cli-plan-before-you-build-steer-as-you-go/?utm_source=chatgpt.com

21 <https://github.com/github/copilot-cli/issues/1675>

<https://github.com/github/copilot-cli/issues/1675>

24 26 45 63 67 69 [Getting started with GitHub Copilot CLI - GitHub Docs](https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-getting-started)

<https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-getting-started>

29 30 61 <https://docs.github.com/copilot/how-tos/copilot-cli/administer-copilot-cli-for-your-enterprise>

<https://docs.github.com/copilot/how-tos/copilot-cli/administer-copilot-cli-for-your-enterprise>

33 66 72 [Using hooks with Copilot CLI for predictable, policy-compliant execution - GitHub Docs](https://docs.github.com/en/copilot/tutorials/copilot-cli-hooks)

<https://docs.github.com/en/copilot/tutorials/copilot-cli-hooks>

37 [Requests in GitHub Copilot](https://docs.github.com/en/copilot/concepts/billing/copilot-requests?utm_source=chatgpt.com)

https://docs.github.com/en/copilot/concepts/billing/copilot-requests?utm_source=chatgpt.com

38 40 60 [Using GitHub Copilot CLI - GitHub Docs](https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli)

<https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli>

39 68 [Configure GitHub Copilot CLI - GitHub Docs](https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/configure-copilot-cli)

<https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/configure-copilot-cli>

46 73 75 <https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-with-actions>

<https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-with-actions>

47 64 [Comparing GitHub Copilot CLI customization features - GitHub Docs](https://docs.github.com/en/copilot/concepts/agents/copilot-cli/comparing-cli-features)

<https://docs.github.com/en/copilot/concepts/agents/copilot-cli/comparing-cli-features>

48 **Creating agent skills for GitHub Copilot CLI**

https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/create-skills?utm_source=chatgpt.com

50 **GitHub Copilot CLI plugin reference - GitHub Docs**

<https://docs.github.com/en/copilot/reference/cli-plugin-reference>

51 52 53 54 **Running tasks in parallel with the /fleet command - GitHub Docs**

<https://docs.github.com/en/copilot/concepts/agents/copilot-cli/fleet>

56 **ACP support in Copilot CLI is now in public preview - GitHub Changelog**

<https://github.blog/changelog/2026-01-28-acp-support-in-copilot-cli-is-now-in-public-preview/>

57 **Copilot SDK in technical preview · community · Discussion #184872 · GitHub**

<https://github.com/orgs/community/discussions/184872>

58 **Install and Use GitHub Copilot CLI directly from the GitHub CLI - GitHub Changelog**

<https://github.blog/changelog/2026-01-21-install-and-use-github-copilot-cli-directly-from-the-github-cli/>